

README

Part 13: 数据工程 — 从手写 MinHash 到 Data-Juicer

- > "模型质量的上限是数据质量"——但数据工程是从零课程最常缺失的一环（也是面试簇 G 的空白）。本部分先手写工业去重的核心算法（MinHash + 分带 LSH，~60 行），
- > 再用工业工具 Data-Juicer（阿里，200+ 算子）复跑同一条管线，对照"手写 60 行 vs
- > 工业工程差 4 个数量级"的每一步。
- > 主源：[datajuicer/data-juicer](https://github.com/datajuicer/data-juicer)（7.0k，Apache-2.0，阿里通义）

学习目标

完成本部分后，你将能够：

- 理解 数据工程在 LLM 链路中的位置和价值
- 手写 MinHash + LSH 去重算法（~60 行），解释其概率性质
- 配置 Data-Juicer 的 YAML 管线并理解每个算子的作用
- 设计 一条完整的数据清洗管线
- 识别 常见的数据质量问题并设计解决方案

章节导航

	内容
ling_laws.md) 建议先读	Chinchilla 三项式推导与拟合、最优 N:D、数据重复 R 的折扣、过训练时代（Llama 3 的 1875 t/p）——本 Part 的"预算语言"
重](01_dedup_from_scratch.md)	shingling → 签名 → 分带 LSH → Jaccard 验证，LSH 概率性质
a_juicer_pipeline.md)	YAML 配置驱动、算子全家桶、追踪审计、FineWeb 对照

- > 00 章是开篇：它回答"数据要洗到多净、攒到多少才够"的度量衡问题
- > $(L(N,D)=E+A/N^{\alpha}+B/D^{\beta})$ ，后面 01/02 章的每一刀删减都用它来算账。
- > 跳过 00 直接读 01 也可以，但"去重删多少可以接受"会缺少判断依据。

前置知识

必须掌握：

- [Part 8 · 07 评估学](../Part8_post_training/tutorial/07_evaluation.md)：评估学里的"污染去重"（n-gram 重叠检查——本章是它的算法底座）。为什么需要：先见过"为什么要去重"，本章才回答"怎么去重"。

建议掌握：

- 概率直觉：P[两集合最小哈希相等] = Jaccard（[01 章](01_dedup_from_scratch.md)推导）。为什么需要：MinHash 的全部正确性都建立在这一个等式上。

可选：

- [Part 7 · Minimind 预训练](../Part7_minimind/tutorial/README.md)：预训练流程（了解数据在预训练中的作用）。为什么需要：去重/过滤的收益最终要在预训练 loss 上兑现。

在 LLM 链路中的位置

【本部分: 数据工程】→ 预训练(Part 7) → SFT/对齐(Part 8/11/12) → 部署(Part 14)
↑
你在这里

为什么数据工程是"模型质量的上限"：

证据	说明
FineWeb	用 5-gram MinHash 全局去重 + 质量过滤，拿下当时最佳开源预训练集
Gopher	去重提升基准最高 +1.5%，去污染 +2.6%
Llama 3	数据质量是模型性能的关键因素之一

理论背景

问题引入：为什么需要数据工程？

预训练数据虽然"多"，但质量参差不齐：

- 1. 重复数据：同一文档出现多次，导致模型"记住"而非"理解"
- 2. 低质量数据：垃圾邮件、广告、机器生成的内容
- 3. 污染数据：测试集泄露到训练集，导致评估失真

数据工程通过清洗、去重、过滤来提升数据质量：

原始数据: "大量但质量参差不齐"

↓ 去重

去重后: "去除重复，减少记忆"

↓ 过滤

过滤后: "去除低质量，保留高质量"

↓ 混合

混合后: "平衡不同领域，提升泛化能力"

- > 类比：原始数据像是未经筛选的食材，数据工程像是洗菜、切菜、调味。
- > 食材质量决定了菜品的上限。

数学推导：MinHash 的概率性质

MinHash 的核心思想是：用随机哈希函数近似 Jaccard 相似度。

问题设定：

- 两个集合 A 和 B
- Jaccard 相似度： $J(A,B) = |A \cap B| / |A \cup B|$

推导过程：

Step 1: 定义 MinHash

$h(S) = \min\{x \in S \mid h(x)\}$ # 集合 S 中哈希值最小的元素

Step 2: 概率性质

$P[h(A) = h(B)] = J(A, B)$

证明：

- 设 $U = A \cup B$
- 对于 U 中的任意元素 x , $h(x)$ 是随机的
- $h(A) = h(B)$ 当且仅当 $A \cup B$ 中哈希值最小的元素在 $A \cap B$ 中
- $P[\min \in A \cap B] = |A \cap B| / |A \cup B| = J(A, B)$

Step 3: 多次哈希估计

使用 k 个独立的哈希函数 $h_1, h_2, \dots, h_k$

签名向量： $\text{sig}(A) = [h_1(A), h_2(A), \dots, h_k(A)]$

估计 Jaccard： $J_{\text{est}} = (1/k) * \sum I[h_i(A) = h_i(B)]$

性质：

- $E[J_{\text{est}}] = J(A, B)$ (无偏估计)
- $\text{Var}[J_{\text{est}}] = J(A, B) * (1 - J(A, B)) / k$
- k 越大, 估计越准

关键洞察：

- MinHash 把集合相似度问题转化为向量比较问题
- 签名向量的维度 k 控制估计精度
- 工业上 $k=128-256$ 是常见配置

历史脉络：数据工程演进

2018: 精确去重 (hash-based)

↓ $O(n^2)$ 无法扩展

2020: MinHash + LSH (近似去重)

↓ 近线性复杂度

2022: 质量过滤 (perplexity, classifier)

↓ 自动化过滤

2024: FineWeb (5-gram MinHash + 质量过滤)

↓ 工业级数据工程

关键论文：

- MinHash: [Similarity Estimation Techniques from Rounding Algorithms](<https://cs.brown.edu/research/pubs/theses/ugrad/2005/broder.pdf>)
- FineWeb: [The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale](<https://arxiv.org/abs/2406.17557>)
- Gopher: [Scaling Language Models: Methods, Analysis & Insights from Training Gopher](<https://arxiv.org/abs/2112.11446>)

环境与版本策略

bash

01 章手写：无需任何安装（纯 Python 标准库）

02 章 Data-Juicer：跟随 latest，文本管线 CPU 即可

pip install py-data-juicer # 重依赖 (Ray/多模态) 为可选 extras, 按需装

你有什么	能做什么
任何机器 (含 CPU 笔记本)	全部内容——01 手写 + 02 Data-Juicer 小管线

学习地图

Scaling Law 开篇 (00: 数据要攒多少才够的度量衡) ← 预算语言

↓ "删多少、重复几遍, 用 $L(N,D)$ 算账"

手写 MinHash/LSH (01: 数学+实现) ← 点

↓ "这 60 行被工业版怎么放大?"

Data-Juicer YAML 管线 (02) ← 面 (200+ 算子、审计、分布式)

↓ 读 FineWeb/Gopher 的真实做法

自己设计一条清洗管线 → 面试/工作就绪

课后作业

每章末尾有思考题 (折叠答案)。全部学完后:

[Assignment 13](../..../assignments/assignment_13/)

相关资源

- [Data-Juicer](https://github.com/datajuicer/data-juicer) · [data-juicer-hub](https://github.com/datajuicer/data-juicer-hub) (50+ 配方)
- [FineWeb 博客](https://huggingface.co/spaces/HuggingFaceFW/blogpost-fineweb-v1) (工业去重的最佳叙述: 5-gram、14 band $\times$ 8 row、阈值 ≈ 0.7)
- [Deduplicating Training Data Makes LMs Better](https://arxiv.org/abs/2107.06499) · Gopher (arXiv 2112.11446)

[← 上一章: Part 12 LLaMA-Factory](../..../Part12_finetune_llamafactory/tutorial/README.md) | [下一章: Part 14 vLLM →](../..../Part14_inference_vllm/tutorial/README.md)

00_scaling_laws

00 — Scaling Law 开篇: 数据要攒多少才够?

- > Part 13 的主题是"把数据洗到多干净、攒到多少才够"。但"够"字需要一个度量衡:
- > 删掉 10% 的语料值多少 loss? 多攒 5B token 能换几个点? 这些问题在 Scaling Law
- > 出现之前只能拍脑袋。本章是这个 Part 的开篇 (建议先读, 再进 01 章手写
- > MinHash): 我们把 Chinchilla 的
- > $L(N,D) = E + A/N^\alpha + B/D^\beta$ 从"背诵常数"变成"亲手拟合"

- > (跑 [scripts/00_scaling_laws.py](../scripts/00_scaling_laws.py) 的三种模式) ,
- > 让后面所有的去重/过滤决策都有一条公式来算账。

学习目标

完成本章后，你将能够：

- 推导 Chinchilla 三项式：从 Kaplan 幂律出发，逐步推出 $L(N,D)=E+A/N^\alpha+B/D^\beta$ 并解释三个参数各自的物理含义
- 拟合：用 Huber 损失 + `scipy.optimize` 把一组 (N, D, final_loss) 记录拟合成五参数 (fit_chinchilla)，并知道什么网格设计会让拟合病态
- 解释过训练 (over-training) 与数据约束：为什么 $R \leq 4$ 个 epoch 的重复近似免费、 $R > 16$ 收益递减趋向于零，以及 Llama 3 为什么敢把 8B 模型训到 1875 t/p
- 设计数据预算：给定算力/模型规模，用 t/p (tokens per parameter) 语言决定"该攒多少数据、去重删多少可以接受"

前置知识

必须掌握：

- [Part 7 · 预训练流程](../Part7_minimind/tutorial/README.md)：知道预训练在训什么 (next-token prediction、cross-entropy loss)。为什么需要：本章的所有 final_loss 就是这个量。

建议掌握：

- [Part 8 · 01 GPT 与预训练](../Part8_post_training/tutorial/01_gpt_and_pretrain.md)：本章 scan/epoch 模式自建的 1M~6M 参数小 GPT 就是它的缩小版 (LayerNorm + learned PE + MHA + ReLU FFN 经典款)。
- 对数坐标与幂律： $\log y = a - b \cdot \log x$ 的直线化。

可选：

- 微积分 (Lagrange 乘法法)：最优 N:D 配比推导用到，跳过证明直接用结论也行。

问题引入：数据洗到多干净、攒到多少才够？

Part 13 要做的事情很清楚：去重、过滤、混料。但每一道工序都在删数据 (FineWeb 的 5-gram 去重 + 质量过滤就是这样的流水线)：

- MinHash 去重删掉一档规模
- 质量过滤器再删一档
- 最后你要问：删掉的这些，值多少 loss？剩下的语料要训多少 epoch？

没有 Scaling Law 时的决策方式：拍脑袋 + 消融实验烧钱。有的企业真的为"该训 1 个 epoch 还是 4 个 epoch"各烧一次训练来对比。有了 Scaling Law：

$\text{loss} = E + A/N^\alpha + B/D^\beta \leftarrow$ 一条公式刻画 (参数, 数据, loss) 三者关系

↓ 推导

给定算力 $C=6ND$ ，最优的 N 和 D 各是多少？(→ 本章数学推导)

↓ 推广

数据重复 R 次 $\approx$ 什么效果？(→ epoch 模式实测)

- > 类比：Scaling Law 是预训练的"物价表"。去重就像把菜里的烂叶子摘掉——
- > 摘掉多少可接受，得先知道一斤好菜值多少钱（E 和 B/D^β ），以及烂叶子
- > 本来值多少（重复数据在 $R>4$ 后的折扣）。
- > 关键概念 t/p（tokens per parameter）= D/N：训练 token 数除以参数量。
- > Chinchilla 结论 ≈ 20 t/p；Llama 3-8B 是 1875 t/p。整个"该攒多少数据"
- > 的讨论都可以压缩成这一个数字。

历史脉络

	论文
幂律：loss 随 N、D、C 各自成幂律下降；建议优先扩参数	[2001.08361](https://arxiv.org/ab
chilla：修正 Kaplan，N 和 D 应等比例扩，最优 ≈ 20 t/p；70B/1.4T 同算力反超 Gopher（论文原文 "70B parameter model"）	[2203.15556](https://arxiv.org/ab
anighoff：数据受限下的 scaling——重复 4 个 epoch 内近似免费，16 后趋零	[2305.16264](https://arxiv.org/ab
ffer：涌现能力可能是指标的"海市蜃楼"	[2304.15004](https://arxiv.org/ab
oglu 等：Chinchilla 复现再分析 + inference-aware 修正	[2401.00448](https://arxiv.org/ab
a 3：8B 模型 15T token（1875 t/p），"过训练"成为主流	[2407.21783](https://arxiv.org/ab
wski：MoE 的细粒度（granularity）成为新 scaling 变量	[2402.07871](https://arxiv.org/ab

数学推导

第 1 步：Kaplan 幂律（单变量）

Kaplan et al.（2020）的实验观察：固定其他因素，loss 随参数量 N 成幂律：

$$L(N) = E + A \cdot N^{(-\alpha)}$$

两边取对数（减去 E 后）：

$$\ln(L - E) = \ln A - \alpha \cdot \ln N \leftarrow \text{在 log-log 图上是一条直线}$$

- E：不可约损失（irreducible loss）——数据的内在熵，参数再多也压不下去
- A：尺度系数； α ：幂指数（Kaplan 的 N 指数远小于 Chinchilla 后来拟合的 $\alpha=0.34$ ——这个差异正是两篇论文结论冲突的源头之一）

- > 为什么会有幂律？理论解释至今没有定论（spectra、神经正切核、随机矩阵
- > 都给过解释）。工程上的态度：当经验规律用，但要知道它的适用边界。

对 D（数据）和 C（算力）有完全对称的形式： $L(D)=E+B \cdot D^{(-\beta)}$ 、
 $L(C)=E+...C^{(-\gamma)}$ 。Kaplan 把 C 的指数做外推后得出"优先扩参数"的结论。

第 2 步：算力怎么算——6ND 的来历

一次 forward：每个参数大约参与 2 次 FLOPs（一次乘一次加） $\times N$ 个参数 $\times D$ 个 token $\approx 2ND$ 。backward 要算梯度，约为 forward 的 2 倍 $\approx 4ND$ 。合计：

$$C \approx 6ND \text{（FLOPs；忽略 attention 的 } O(T \cdot d) \text{ 项，长上下文时要修正）}$$

这就是所有 isoFLOP 分析的"预算约束"。

第3步：Chinchilla 三项式（双变量）

Kaplan 的单变量幂律有个隐患：扫 N 时 D 固定（或扫 D 时 N 固定），两项误差互相污染。Chinchilla (Hoffmann et al. 2022) 直接给出联合形式：

$$L(N, D) = E + A/N^\alpha + B/D^\beta$$

参数	Chinchilla 拟合值	含义
E	1.69	不可约损失（数据条件熵下界）
A	406.4	容量项系数：模型不够大要付的代价
α	0.34	容量项指数
B	410.7	数据项系数：token 不够多要付的代价
β	0.28	数据项指数

(来源：2203.15556 Table 3 的参数化拟合；注意 N 是非 embedding 参数。)

三项的直觉：

loss

Diagram illustrating the relationship between Entropy (E), Data (B/D), Capacity (A/N), and the Number of Tokens (N, D):

- Top row: E ————— 熵下界：谁也压不过去
- Second row: B/D $\wedge$ β 数据项：多看 token 就降
- Third row: A/N $\wedge$ α 容量项：模型大就降
- Bottom row: ————— $\rightarrow$ N, D

第4步：最优配比 (Lagrange 推导, 本教程的核心公式)

问题：给定算力预算 C ，怎么分给 N 和 D ？

目标： $\min L(N,D) = E + A \cdot N^{-\alpha} + B \cdot D^{-\beta}$ ，约束 $C = 6ND$ 。

把 $D = C/(6N)$ 代入，对 N 求导置零：

$$d/dN [A \cdot N^{(-\alpha)} + B \cdot (6N/C)^{(\beta)}] = 0$$

$$-\alpha \cdot A \cdot N^{(-\alpha-1)} + \beta \cdot B \cdot (6/C)^{\beta} \cdot N^{(\beta-1)} = 0$$

↓ 移项

$$\alpha \cdot A \cdot N^{(-\alpha)} = \beta \cdot B \cdot D^{(-\beta)} \leftarrow \text{两个代价项在边际上相等 (经济学直觉)}$$

↓ 解出

$$N_{opt} = (\alpha A / \beta B)^{1/(\alpha + \beta)} \cdot (C/6)^{\beta/(\alpha + \beta)}$$

$$D_{opt} = (\beta B / \alpha A)^{1/(\alpha + \beta)} \cdot (C/6)^{\alpha/(\alpha + \beta)}$$

> 关键洞察： $\alpha \cdot A \cdot N^{(-\alpha)} = \beta \cdot B \cdot D^{(-\beta)}$ 是"边际收益相等"条件——

> 再花 1 FLOPs 在扩参数上省的 loss = 花在加数据上省的 loss。

> 与经济学中"预算约束下的最优消费组合"完全同构。

代入 Chinchilla 参数, N、D 随 C 的指数分别是 $\beta/(\alpha+\beta)\approx 0.45$ 和

 $\alpha/(\alpha + \beta) \approx 0.55$ ——近似等比例增长, $t/p \approx 20$ 且随 C 缓慢变化。

这就是 --mode fit 输出里 t/p 从 11 (C=1e14) 缓慢爬到 21.5 (C=1e17) 的来源。

> ⚠ Kaplan vs Chinchilla 到底差在哪 (LR horizon 的坑) : Kaplan 的一部分

- > 训练没有把每个 (N,D) run 的学习率调度调到该 run 自己的 token 预算
- > ——小/短的 run 相当于被提前掐断，loss 被系统性高估，于是结论偏向"数据
- > 不重要、堆参数"。Chinchilla 的修正之一就是逐 run 设置 cosine horizon。
- > Besiroglu et al. (2401.00448) 的再分析进一步指出 Kaplan 的三参数拟合
- > 存在方法学问题。这不是历史八卦：你自己的 scan 实验里，如果所有 run
- > 共用同一条 LR schedule，会得到一模一样的偏差（脚本 lr_at() 的注释
- > 就是这条红线）。

第 5 步：数据约束——重复 R 次值多少？

上面全部假设"数据无限多"。现实里 unique 语料是有限的，只能重复：

$R = D / D_u$ （D = 训练 token 总数，D_u = unique token 数，R = epoch 数）

Muennighoff et al. (2305.16264) 的系统实验结论（本课 --mode epoch 复现）：

- $R \leq 4$ ：重复数据近似等于新鲜数据（loss 沿"新鲜数据幂律"继续下降）
- $4 < R \leq 16$ ：边际收益递减，重复 token 开始"打折"
- $R > 16$ ：收益递减、趋向于零——论文措辞是"4 epoch 内近似等价于新鲜数据，更多重复收益递减"

公式化描述：把幂律里的 D 换成"有效 token" D，D 随 R 增长但饱和（R 大时 $D^* \rightarrow$ 常数）。epoch 模式打印的 R_eff/R（折扣）列就是它的实测版。

第 6 步：过训练时代——为什么 Llama 3 敢用 1875 t/p

Chinchilla 的"最优"只算训练算力。但模型训完要部署推理：8B 推一次的成本 $\approx$ 每 token 2N FLOPs，部署期的总推理 FLOPs 往往远超训练。

模型	参数	训练 token	t/p	相对 Chinchilla 最优 (~20)
Chinchilla	70B	1.4T	20	1 $\times$ （基准）
Llama 2-70B	70B	2T	≈ 29	1.4 $\times$ 过训练
Llama 3.1-8B	8B	15T	≈ 1875	$\approx 90 \times$ 过训练

（Llama 3 数据见 2407.21783；8B 的 Chinchilla 最优约 1600~2000 亿 token。）

逻辑链：推理成本主导 $\rightarrow$ 同等总成本下，小模型+多数据比大模型+少数据便宜 $\rightarrow$ 最优 t/p 远大于 20。Besiroglu et al. (2401.00448) 把这个逻辑形式化为 inference-aware scaling：把部署期推理 FLOPs 计入目标函数，解出来的 N 明显更小、D 明显更大。

- > 这正是数据工程在 2024 年后突然更重要的原因：模型小了，数据配额变大
- > 几十倍，去重/过滤/混料的每一个决策都被放大。

附：涌现能力之争（衔接 Part 8 评估学）

Scaling law 说 loss 平滑下降，那"模型到某个规模突然会做数学/推理"是怎么回事？

- Schaeffer et al. ([2304.15004](https://arxiv.org/abs/2304.15004))：
- 许多"涌现"是指标选择造成的假象——用不连续指标（如 exact match 全对才算分）时，平滑增长的 capability 会被折成阶跃；换成连续指标（如 token 级概率）曲线是平滑的。
- Jason Wei（涌现综述 2206.07682 的一作）的

[博客回应](https://www.jasonwei.net/blog/common-arguments-regarding-emergent-abilities)：
指标 artifact 的论证不充分——即使在连续指标下，部分能力的增长斜率
仍随规模显著变陡；且"对用户而言可感知的阈值"有工程意义。

> 这场争论的实用教训与 [Part 8 · 07 评估学](../Part8_post_training/tutorial/07_evaluation.md)
> 直接相关：你汇报的"能力跳变"可能只是你选的指标在跳变。选指标前先问
> 它是连续的还是阶跃的。

附：MoE 的 scaling——粒度作为新变量（衔接 Part 7）

MoE 把"参数量 N"拆成两件事：总参数（所有专家）与每次前向激活的参数。
Krajewski et al. ([2402.07871](https://arxiv.org/abs/2402.07871)) 系统扫描了
专家粒度（granularity，总参数固定时切多少个专家）：细粒度 MoE 在相同
训练算力下 loss 更优——因为 6ND 里的"有效 N"应按激活参数算，而表达能力
随专家数提升。所以在 MoE 语境下读 scaling law 时要问一句：公式里的 N
是总参数还是激活参数？（[Part 7 · 03 章](../Part7_minimind/tutorial/03_gqa_and_ffn.md)
手写过 MoE，可对照。）

代码实现

[scripts/00_scaling_laws.py](../scripts/00_scaling_laws.py)，三种模式：

```
bash
python 00_scaling_laws.py --mode fit # 零 GPU，~2s：合成数据拟合 + isoFLOP 图
python 00_scaling_laws.py --mode scan # 单卡 ~25s：网格真训小 GPT + 自己的 scaling law
python 00_scaling_laws.py --mode epoch # 单卡 ~31s：固定语料 × R epoch 饱和实验
```

两个跨脚本复用的接口（后续复现/论文核对会用到，签名稳定便于复用）：

```
python
def chinchilla_loss(N, D, params):
    """L(N,D) = E + A/N^alpha + B/D^beta ; params=(E,A,alpha,B,beta)"""

def fit_chinchilla(records, n_starts=8, huber_delta=0.05, seed=13, E_fixed=None):
    """records = [(N, D, final_loss), ...] → (E, A, alpha, B, beta)
    Huber + scipy.least_squares, A/B 用 log 参数化, 8 起点随机重启"""
```

形状追踪：scan/epoch 模式的数据通路

```
token 池 (n_tokens,) uint8 # scan: 马尔可夫重采样；epoch: input.txt
| ChunkSampler / EpochLoader：切成互不重叠 chunk (B×T=8192 token)
↓ view(B, T)
seq (B=32, T=256)
| x = seq[:, :-1] y = seq[:, 1:] # 因果错位：预测下一个 token
↓
x, y (32, 255) int64
| token_embed + pos_emb → (32, 255, d)
| L × Block (Pre-LN + MHA + FFN) # 形状不变
| ln_f → lm_head → (32, 255, V)
↓ cross_entropy(logits.reshape(BT, V), y.reshape(BT))
```

loss 标量 (nat/token)

关键实现点 (都有坑, 注释在现场) :

1. LR horizon 逐 run 设置 (lr_at 的注释) : cosine 的 total_steps = 本 run 的 token 预算 $\div$ 每步 token 数。这是 Kaplan 偏差的直接对策。
2. Huber 而非最小二乘 : 真实网格总有坏点, Huber 在 $|r| > \delta$ 后线性化, 坏点自动降权。
3. 多起点 : 幂律拟合非凸, 8 次随机重启取代价最小者。
4. B 取小换步数 (B=32, T=256) : 同量 token 下更多优化步——开发时实测 B=256 时步数太少, 模型连"掉出均匀分布盆地"都来不及。

实测 (三模式真实输出)

> 环境标注 : RTX 4090 (24GB) 单卡 · torch 2.6.0+cu124 · Python 3.12 ·
> scipy 1.18.1 · bf16 autocast。以下输出均为脚本真实运行结果 (非编造)。

模式一 : fit —— 16 次噪声抽取平均后全部 <5%

text

[1] 合成网格: 64 个 (N, D) 点 $\times$ 16 次独立 3% 噪声抽取
真值来自 Hoffmann 2203.15556 Table 3: E=1.69 A=406.4 α =0.34 B=410.7 β =0.28

[2] 单次噪声抽取的拟合结果 (看方差, 不验收; 截选)
A: 真值 406.400 单次拟合 360.954 偏差 +11.2% $\leftarrow$ 单次会偏 10%+

[3] 16 次独立噪声实现 $\rightarrow$ 拟合 $\rightarrow$ 参数平均 (0.4s)

参数 真值 平均拟合 相对误差 跨抽取 std 判定

E 1.690 1.692 0.13% 0.052 PASS

A 406.400 416.292 2.43% 38.762 PASS

alpha 0.340 0.342 0.53% 0.007 PASS

B 410.700 424.322 3.32% 69.751 PASS

beta 0.280 0.281 0.44% 0.010 PASS

$\rightarrow$ 全部参数相对误差 <5%

[4] isoFLOP 剖面 (用拟合参数画; 谷底 = 该算力预算下的最优 N; 共 7 档 C, 输出截选 4 档)

C (FLOPs) N_opt D_opt D/N (t/p)

1e+14 1.23e+06 1.35e+07 11.0

1e+15 3.48e+06 4.78e+07 13.7

1e+16 9.85e+06 1.69e+08 17.2

1e+17 2.78e+07 5.99e+08 21.5

![isoFLOP 剖面图](../scripts/output_scaling_fit.png)

读图要点 : 每条 U 形曲线是同一算力预算下 loss 随 N 的变化——左边是 "模型太小、算力浪费在数据上", 右边是 "模型太大、数据不够喂"; 谷底 (▼) 随 C 增大右移, 且 t/p 缓慢爬升, 与 Chinchilla 的 ~ 20 t/p 吻合。

> 为什么网格要跨 5 个数量级 (N 从 1e5 到 1e10、D 从 1e7 到 1e12) :

> 让 N 项和 D 项各有 "主导角" 与 "可忽略角", 5 个参数才可辨识。开发首版

> 用了窄网格 (D 项始终占比 <30%), B 的拟合误差 262%——参数沿平坦方向漂移。

> 单次噪声抽取的系数误差天然有 $\pm 10 \sim 15\%$ (跨抽取 std 列), 16 次抽取取平均

> 才能稳定达标。这正是工业 scaling 实验报 "均值 $\pm$ std" 的原因。

模式二：scan —— 3×3 网格真训 + 自拟合 (~25s, 输出截选)

text

[1] 语料池：input.txt 拟合的 3 阶马尔可夫重采样 ('无限唯一数据'区)

池大小: 24.0M token ($\geq$ 最大 D, 单 run 内 token 最多见 1 次)

vocab=65 (char 级) 熵下界 $E=2.616$ nat/token vs 随机猜测 $\ln(65)=4.174$

[3] (N, D, val_loss) 网格表 $\leftarrow$ 双向单调 !

N D t/p val_loss

1.24e+05 2e+06 16.1 2.9687

1.24e+05 6e+06 48.2 2.9438

1.24e+05 2e+07 160.9 2.7942

2.6e+05 2e+06 7.7 2.9543

2.6e+05 6e+06 23.0 2.9268

2.6e+05 2e+07 76.9 2.7699

9.88e+05 2e+06 2.0 2.9426

9.88e+05 6e+06 6.1 2.8267

9.88e+05 2e+07 20.2 2.7575

[4] fit_chinchilla 拟合

自由 5 参数: $E=0.0000$ $A=1.04$ $\alpha=0.100$ $B=4.18$ $\beta=0.031 \leftarrow$ 病态 !

固定 $E=2.616$ (语料熵下界可独立测量——合成语料独有的优势):

$E=2.616$ $A=421.3$ $\alpha=0.742$ $B=52.7$ $\beta=0.354$

相对残差: 均值 +0.02% 最大绝对值 2.08%

[5] 计算最优配比

C (FLOPs) N_opt D_opt D/N (t/p)

1.49e+12 6.32e+04 3.93e+06 62.3

1.18e+13 1.23e+05 1.6e+07 129.6

1.19e+14 2.6e+05 7.6e+07 292.4

学生结论: 本玩具尺度 (char 级、 $\leq 1M$ 参数) 最优 $D/N \approx 62 \sim 292$ t/p

三段式解读:

1. 网格表双向单调——固定 N 加 D 降 loss, 固定 D 加 N 也降 loss。

这是拟合有意义的必要条件 (首版网格 N 太大时 D 方向完全平的)。

2. 自由拟合病态 (E 被顶到 0): 玩具网格的动态范围只有 ~ 0.2 nat,

E 在网格内不可辨识——fit 模式用了 12 个数量级才钉住它。解法:

本课语料是合成的, 熵下界可以直接测量 (2.616), 固定 E 后指数立刻正常

($\alpha=0.74$ 、 $\beta=0.35$, 残差 2%)。

3. 学生结论的解读: 玩具最优 t/p $\approx 62 \sim 292$, 远高于 Chinchilla 的 20——

因为这个任务的可学结构在 $\sim 0.3M$ 参数就饱和了, 多余预算全部流向数据。

N:D 没有普适值, 只有"对给定任务/尺度测量出来的值"——这本身就是

本模式最重要的教学输出。

模式三：epoch —— $R \leq 4$ 近似线性, $R=16$ 饱和+过拟合 (~31s)

text

[1] 真实语料: data/input.txt (tiny shakespeare)

训练 (unique): 1.004M token 验证 (held-out): 112K token

[2] 模型: 6.10M 参数 ($d=288$, $L=6$), $R = [1, 2, 4, 8, 16]$

R=1 D=1.00M train=2.4934 val=2.4866
R=2 D=2.00M train=2.4075 val=2.4337
R=4 D=4.00M train=1.8899 val=1.9676
R=8 D=8.00M train=1.4214 val=1.6988
R=16 D=15.99M train=0.9505 val=1.8849 ← val 反升！

[4] 幂律拟合（仅用 $R \leq 4$ 的点）： $L = 2.567 \cdot R^{-0.169}$

R 实测 val 幂律预测 偏差 判定

1 2.4866 2.5671 -3.1% 线性区内 ✓
2 2.4337 2.2835 +6.6% 线性区内 ✓
4 1.9676 2.0313 -3.1% 线性区内 ✓
8 1.6988 1.8069 -6.0% 未饱和
16 1.8849 1.6073 +17.3% 饱和（实测高于外推）

[5] 有效 token 数

R 名义 D 有效 D_{R_eff}/R （折扣）

1 1.00M 1.21M 1.21x
2 2.00M 1.38M 0.69x
4 4.00M 4.85M 1.21x
8 8.00M 11.57M 1.44x
16 15.99M 6.25M 0.39x ← 重复 token 只值 0.39 个新的
（ $R \leq 8$ 段的 R_{eff} 波动——如 $R=1$ 的 $1.21x > 1$ ——来自 $\pm 3\sim 7\%$ 的幂律拟合残差，属噪声；真正的信号只有 $R=16$ 的 $0.39x$ 折扣，对应上表 +17.3% 的饱和。）

![epoch 饱和曲线](../scripts/output_scaling_epoch.png)

对照 Muennighoff（2305.16264）的结论读数：

- $R \leq 4$ ：实测贴着幂律外推（ $\pm 3\sim 7\%$ ）——重复数据当新鲜数据用，损失很小 ✓
- $R=8$ ：本玩具尺度仍在赚（模型离吃透 1M 语料还远）；论文尺度上这里是边际收益开始打折的位置（我们欠训练更严重，饱和点整体右移——玩具与真实的诚实差异）
- $R=16$ ：实测比外推差 +17.3%，且 val 反升、train/val 分叉
（train 0.95 vs val 1.88）——模型在背语料而不是学语言。这就是去重的价值所在：把 $R=16$ 的预算换成 4 倍 unique 数据，loss 会好得多。

实验设计复盘：开发时踩的三个坑（本教程最值钱的部分）

scan 模式看起来只有 25 秒，但让它“能出双向单调结果”的设计迭代了四轮。
三个坑都值得记住——你自己做 toy 实验时一定会再遇到：

坑 1：随机合成语料 = 学不动（语料坑）

症状：所有 run 的 loss 死死卡在 $\ln(V)$ （随机猜测水平），N、D 加多少都没用。

原因：首版用随机 Dirichlet 转移表造 3 阶马尔可夫语料。纯 3 阶结构没有任何低阶入口（unigram 均匀、bigram 无信号），而 SGD 学 n-gram 统计需要“先学简单再爬复杂”的阶梯——自然语言天然有（先 unigram 再 bigram 再 …），随机表没有。注意力从零自举出“看前 3 个 token”的模式，在几百步内根本爬不出来（我们实测 2441 步纹丝不动）。

解法：在真实文本（input.txt）上拟合 3 阶插值马尔可夫再重采样——

继承真实文本的难度谱（各阶行熵是 Zipf 式平滑分布），又有无限唯一数据。

坑 2：容量不 binding = 没有模型方向梯度（网格坑）

症状：按规格网格 $N \in \{1M, 3M, 10M\}$ 训完，固定 D 时三个模型的 loss 相差 < 0.01 ——拟合出的 $\alpha \approx 0$ ，谷底在网格外面。

原因：这个任务的可学结构 $\sim 0.3M$ 参数就吃下了，1M/3M/10M 全部"够用"，容量项 A/N^α 从未被激活。谷底必须落在网格内，拟合才有意义。

解法：把 N 缩一个数量级到 $\{0.12M, 0.26M, 1M\}$ （脚本注释里有说明）。这也是 Chinchilla 论文强调的：isoFLOP 实验的 (N,D) 候选要覆盖谷底两侧。

坑 3：大 batch = 步数不够（步数坑）

症状：B=256 时 D=2M 的 run 只有 30 步优化，loss 从初始化掉到均匀分布水平就停住。

原因：玩具尺度的 token 预算小，大 batch 把优化步数吃光了。真实大模型实验 batch 也大，但它们的 token 预算大 5 个数量级，步数反而多。

解法：B=32、T=256——同量 token 换 8 倍步数。

- > 三个坑合起来的教训：toy 实验不是"缩小版的真实实验"。缩小规模会
- > 改变优化动力学（步数）、任务难度谱（语料）、以及哪个资源先饱和（网格）。
- > 用 toy 复现经典结论前，先确认这三件事仍然同构。

常见陷阱

陷阱 1：所有 run 共用一条 LR schedule

症状：scan 网格里小 D 的 run loss 明显偏高，拟合出"数据不重要"的结论。

原因：短 run 的 cosine 还没衰减到低点就结束，系统性欠训练——这就是 Kaplan 偏差的机制。

解法：total_steps 逐 run 设置成 $D_{run} / (B \times T)$ （脚本 lr_at()）。

陷阱 2：拟合时不看参数边界

症状：拟合结果里 $E \approx 0$ （顶到下界）或指数顶到 0.01/1.5 的边界值。

原因：网格动态范围不够，参数沿平坦方向漂移到边界（scan 首版的真实经历）。

解法：扩网格跨度；或固定可独立测量的参数（如本课的 E =语料熵下界）。

陷阱 3：用训练 loss 而不是验证 loss 做 scaling

症状：R=16 的"loss"还在下降，得出"重复数据一直有用"的错误结论。

原因：多 epoch 后模型开始记忆语料，训练 loss 反映的是背诵能力。

解法：一律用 held-out 验证 loss（epoch 模式的 val 切片从不参与训练；train/val 分叉本身就是记忆的探测器）。

练习与思考

概念检验

Q1: 为什么 Chinchilla 三项式里的 E 对自然语言是 1.69 nat 而不是 0？

A: E 是数据的条件熵下界：给定前文，下一个 token 的内在不确定性。自然语言本身有随机性（同一个前文可以接多种合理的续写），再大的模型也不可能把 loss 压到 0。本课 scan 语料的 $E=2.616$ 就是这个含义——它是“这门语言从信息论上最便宜的可达 loss”。玩具实验的优势是 E 可以直接测量（生成器知道条件分布），自然语言的 E 只能靠拟合外推。

Q2: Llama 3-8B 用 1875 t/p 训练，比 Chinchilla 最优过训练约 90 倍。这违反 scaling law 吗？

A: 不违反。Chinchilla 的“最优”目标函数是最小化训练算力换 loss；Llama 3 优化的是部署总成本（训练 + 海量推理）。推理成本与参数量成正比（每 token 约 $2N$ FLOPs），所以把 N 缩小、 D 放大，虽然训练算力“浪费”了，推理便宜了几十倍。Besiroglu et al. (2401.00448) 的 inference-aware 分析就是这个逻辑的形式化。另外过训练的模型在同参数量下更好，蒸馏时也更值钱。

Q3: 你的 scan 网格里，固定 $C=6ND$ ，把 (N,D) 从 $(1M, 20M)$ 改成 $(4M, 5M)$ ——loss 会怎么变？为什么这能用来找谷底？

A: 沿同一条 isoFLOP 线移动： $(1M, 20M)$ 的 $t/p=20$ 偏“数据侧”（模型偏小，容量项 A/N^α 偏大）， $(4M, 5M)$ 的 $t/p=1.25$ 偏“参数侧”（数据项 B/D^β 偏大）。loss 先降后升，最低点就是该算力下的最优配比——这正是 isoFLOP 实验找谷底的原理（fit 模式的 U 形曲线族）。实际操作要跑同一 C 下的多个 (N,D) 候选。

动手实践

练习 1: 扩网格重拟合

任务：跑 `--mode scan --full`（ N 到 3M、 D 到 60M 的 4×3 网格，约 3 倍 smoke 时间），把新 records 喂给 `fit_chinchilla`（记得对比自由拟合与 $E_{\text{fixed}}=H_{\text{floor}}$ 两种）。

验收标准：

- [] 网格表在 N 、 D 两个方向都单调
- [] 汇报 E 固定拟合的 (α, β) 与 smoke 版（ $\alpha=0.742, \beta=0.354$ ）的差异
- [] 解释：网格变大后，最优 t/p 是升高还是降低？为什么？

步骤提示：

python

脚本已经支持：`python 00_scaling_laws.py --mode scan --full`

想自己调网格：改 `run_scan_mode` 里的 `N_targets / D_targets`,

`records = [(N, D, val_loss), ...]` 直接喂 `fit_chinchilla(records, E_fixed=2.616)`

练习 2: 加数据 vs 多 epoch, 谁划算?

任务: 基于 epoch 模式的实测曲线回答: 你有 8M token 的训练预算和 1M 的 unique 语料, (a) 重复 8 遍; (b) 去重放松一点攒到 8M unique (假设幂律 $L=2.567 \cdot R^{(-0.169)}$ 对新鲜数据近似成立)。哪个 loss 更低?

验收标准:

- [] (b) 用幂律外推 $D=8M$ 新鲜数据的 loss ($R_{\text{eff}}=8$)
- [] (a) 用实测 $R=8$ 的 val loss (1.6988) 对比
- [] 一句话结论 + 指出这对去重策略意味着什么

步骤提示:

python

`a = 2.567; gamma = 0.169`

`fresh_8M = a * 8 (-gamma) # (b) 新鲜数据幂律外推`

`repeat_8 = 1.6988 # (a) epoch 模式实测`

对比并解释: 差距就是"重复折扣", 也是去重的价值上限

扩展思考

- 本课 scan 的最优 $t/p \approx 62 \sim 292$, Chinchilla 是 ~ 20 , Llama 3 是 1875—— t/p 由什么决定? (提示: 任务有效复杂度、模型尺度、推理成本三者)
- 如果把 scan 语料换成"代码", 你预期 α 、 β 怎么变? (代码的 n-gram 熵更低、结构更规则)
- Muennighoff 的结论在你的生产场景里怎么用: unique 数据固定时, "训几个 epoch 收手"应该怎么定?

学完本章你能...

- 手推 Chinchilla 三项式与最优配比公式 (Lagrange 两条线)
- 用 `chinchilla_loss / fit_chinchilla` 拟合自己的 scaling 数据, 并诊断病态拟合 (参数顶边界、E 不可辨识)
- 解释 $20 t/p \rightarrow 29 t/p \rightarrow 1875 t/p$ 的演化逻辑 (训练最优 $\rightarrow$ 推理感知)
- 用 $R \leq 4$ 近似免费 / $R > 16$ 趋零的结论, 回答"去重删多少、重复几遍"的预算问题
- 避开 toy scaling 实验的三个坑 (语料难度谱、网格容量 binding、batch 换步数)

参考资源

- Hoffmann et al. 2022, Training Compute-Optimal Large Language Models (Chinchilla) [arXiv 2203.15556](<https://arxiv.org/abs/2203.15556>)
- Kaplan et al. 2020, Scaling Laws for Neural Language Models [arXiv 2001.08361](<https://arxiv.org/abs/2001.08361>)
- Muennighoff et al. 2023, Scaling Data-Constrained Language Models [arXiv 2305.16264](<https://arxiv.org/abs/2305.16264>)

- Besiroglu et al. 2024, Chinchilla Scaling: A replication attempt [arXiv 2401.00448](https://arxiv.org/abs/2401.00448)
- Schaeffer et al. 2023, Are Emergent Abilities of LLMs a Mirage? [arXiv 2304.15004](https://arxiv.org/abs/2304.15004) · Jason Wei 的[博客回应](https://www.jasonwei.net/blog/common-arguments-regarding-emergent-abilities)
- Krajewski et al. 2024, Scaling Laws for Fine-Grained Mixture of Experts [arXiv 2402.07871](https://arxiv.org/abs/2402.07871)
- Lilian Weng, Scaling Laws, Carefully [博客](https://lilianweng.github.io/posts/2026-06-24-scaling-laws/)
- Stanford CS336 Language Models from Scratch (scaling laws 专题) [课程主页](https://stanford-cs336.github.io/)
- datablations (小模型数据消融的公开实验集) [HF 主页](https://huggingface.co/datablations)

下一步

有了预算语言，现在可以回答 Part 13 的第一个工程问题：语料里有多少重复、怎么把它们找出来删掉？下一章手写工业去重的核心算法（MinHash + 分带 LSH, ~60 行纯标准库）。

[01 — 手写 MinHash + LSH 去重](01_dedup_from_scratch.md)

01_dedup_from_scratch

01 — 手写 MinHash + 分带 LSH 去重

- > 去重是数据工程的核心工序：C4 约 4.3% 是精确重复，近似重复更多；去重被证明能同时
- > 提升质量与训练效率（Lee et al. 2021：去重后达到同 loss 的步数显著减少）。
- > 本章手写工业去重的完整算法（跑 [scripts/01_minhash_dedup.py](../scripts/01_minhash_dedup.py),
- > 纯标准库，CPU 几秒）。
- >
- > 前读指引：如果还没读 [00 章 Scaling Law 开篇](00_scaling_laws.md)，建议先花
- > 二十分钟过一遍——“去重删掉的数据值多少 loss、剩下的语料训几个 epoch 划算”
- > (R>4 后重复 token 急剧打折) 正是本章每一刀删减背后的算账依据。

学习目标

完成本章后，你将能够：

- 手写 MinHash + LSH 去重算法（~60 行）
- 解释 MinHash 的数学原理和概率性质
- 画出 LSH 的分带机制和召回率曲线
- 设计去重管线的参数（band、row、阈值）

前置知识

必须掌握：

- Jaccard 相似度： $J(A,B) = |A \cap B| / |A \cup B|$

• 暴力法为什么不可行：N 篇文档要 $O(N^2)$ 次两两比较——十亿级语料下等于不可算，这正是 MinHash+LSH 存在的理由（先"疑似对"再精算，把 $O(N^2)$ 压到近线性）

理论背景

问题引入：为什么需要去重？

预训练数据虽然"多"，但存在大量重复：

1. 精确重复：同一文档出现多次（C4 约 4.3%）
2. 近似重复：同一文档的不同版本（如不同网站转载）
3. 负面影响：模型"记住"而非"理解"，导致过拟合

> 类比：重复数据像是同一道菜反复吃。吃 10 遍同一道菜不会让你成为美食家，> 只会让你对这道菜产生偏见。

数学推导：MinHash 的概率性质

问题设定：

- 两个集合 A 和 B
- Jaccard 相似度： $J(A,B) = |A \cap B| / |A \cup B|$

推导过程：

Step 1: 定义 MinHash

$h(S) = \min\{x \in S \mid h(x)\}$ # 集合 S 中哈希值最小的元素

Step 2: 概率性质

$P[h(A) = h(B)] = J(A,B)$

证明：

- 设 $U = A \cup B$
- 对于 U 中的任意元素 x, $h(x)$ 是随机的
- $h(A) = h(B)$ 当且仅当 $A \cup B$ 中哈希值最小的元素在 $A \cap B$ 中
- $P[\min \in A \cap B] = |A \cap B| / |A \cup B| = J(A,B)$

Step 3: 多次哈希估计

使用 k 个独立的哈希函数 $h_1, h_2, \dots, h_k$

签名向量： $\text{sig}(A) = [h_1(A), h_2(A), \dots, h_k(A)]$

估计 Jaccard： $J_{\text{est}} = (1/k) * \sum I[h_i(A) = h_i(B)]$

性质：

- $E[J_{\text{est}}] = J(A,B)$ （无偏估计）
- $\text{Var}[J_{\text{est}}] = J(A,B) * (1 - J(A,B)) / k$
- k 越大，估计越准

数学推导：LSH 的分带机制

LSH (Locality-Sensitive Hashing) 的核心思想是：把签名向量分成多个 band，只要有一个 band 完全相同就认为是候选对。

问题设定：

- 签名向量维度： k
- 分成 b 个 band，每个 band 有 r 行 ($k = b \times r$)

推导过程：

Step 1: 分带

$\text{sig}(A) = [h_1(A), h_2(A), \dots, h_k(A)]$

分成 b 个 band：

$\text{band}_1 = [h_1(A), \dots, h_r(A)]$

$\text{band}_2 = [h_{r+1}(A), \dots, h_{2r}(A)]$

...

$\text{band}_b = [h_{(b-1)r+1}(A), \dots, h_k(A)]$

Step 2: 候选对判定

A 和 B 是候选对，当且仅当存在某个 band i ，使得 $\text{band}_i(A) = \text{band}_i(B)$

Step 3: 概率分析

$P[\text{某个 band 相同}] = J(A, B)^r$

$P[\text{至少一个 band 相同}] = 1 - (1 - J(A, B)^r)^b$

性质：

- 当 $J(A, B)$ 高时， $P[\text{候选}]$ 接近 1（高召回）
- 当 $J(A, B)$ 低时， $P[\text{候选}]$ 接近 0（低误报）
- b 和 r 控制"召回-误报"的权衡

关键洞察：

- b 越大，召回率越高（更容易找到相似对）
- r 越大，误报率越低（更严格筛选）
- 实践中 $b=14, r=8$ 是 FineWeb 的配置（等效阈值 ≈ 0.7 ）

代码实现

四阶段管线（60 行手写）

运行 `[scripts/01_minhash_dedup.py](../scripts/01_minhash_dedup.py)` 验证以下步骤。

python

① shingling：文档 $\rightarrow$ 3-gram 词组集合 # Jaccard 的比较单位

② MinHash：64 个哈希函数，每个取集合的最小哈希 $\rightarrow$ 64 维"签名"

数学核心： $P[\text{两集合某维最小哈希相等}] = \text{Jaccard}$

③ 分带 LSH：签名切 16 段 $\times$ 每段 4 维；任何一段完全相同 $\rightarrow$ 候选对

概率性质： $P(\text{成为候选}) = 1 - (1 - J^r)^b \# J$ 高 $\rightarrow$ 必然命中

④ Jaccard 验证：只对候选对精算， ≥ 0.5 判重

形状追踪：MinHash 签名过程

| MinHash 签名过程 |

| |

```
| 输入文档: "the cat sat on the mat" |
| ↓ shingling (3-gram) |
| 集合 S = {"the cat sat", "cat sat on", "sat on the", "on the mat"} |
| ↓ 64 个哈希函数 |
| sig(S) = [h_1(S), h_2(S), ..., h_64(S)] |
| = [min(h_1(x) for x in S), min(h_2(x) for x in S), ...] |
| ↓ 分带 LSH (16 bands × 4 rows) |
| band_1 = [h_1(S), h_2(S), h_3(S), h_4(S)] |
| band_2 = [h_5(S), h_6(S), h_7(S), h_8(S)] |
| ... |
| band_16 = [h_61(S), h_62(S), h_63(S), h_64(S)] |
| |
| 候选对判定: 任意 band 完全相同 → 候选对 |
```

实测输出（脚本 01，纯标准库，CPU <1s，开发机实测）：

```
[1] 暴力 Jaccard（真值）：2 对: [('doc00', 'dupA'), ('doc09', 'dupD')]
[2] LSH 候选对：3 对: [('doc00', 'dupA'), ('doc03', 'dupC'), ('doc09', 'dupD')]
[3] LSH 候选 + Jaccard ≥ 0.5：2 对: [('doc00', 'dupA'), ('doc09', 'dupD')]
[4] 性质: 签名一致率 0.69 ≈ 真实 Jaccard 0.73 （P[minhash 相等] = Jaccard，64 维采样）
[5] 去重结果: 14 → 12 篇（丢弃 ['dupA', 'dupD']）
```

• 四个必须理解的点：

- ① $P[\text{候选}] = 1 - (1 - J^r)^b$ —— b/r 是“召回-误报”的调节旋钮（FineWeb：5-gram、14 band × 8 row，等效阈值 ≈ 0.7 ，3.3M CPU 小时跑全网）；
- ② 候选 3 对 → 验证后 2 对：多出来的 ('doc03', 'dupC') 是 LSH 的候选误报，而“被验证步挡掉”正是管线设计的点睛之笔。这对的真实 Jaccard 只有 0.45（阈值 0.5 之下），但 64 维签名的采样一致率是 0.53——有限样本波动让它碰巧撞上了一个完全相同的 band，于是成了候选。LSH 的设计哲学就是用高召回换误报：宁可多报、绝不漏报（对照 [1]，真重复 2 对零漏召），然后把确定性交给第④步 Jaccard 精算兜底（对照 [3]，最终结果零误报）。“宽进严出”两段合起来，才是这条管线的正确性来源——如果删掉验证步，doc03/dupC 这类误报就会直接误删语料；
- ③ 重改写的同义文档 dupB ($J \approx 0.24$) 连候选都不是——它是“语义重复”而非“字面重复”，那是语义去重（embedding 聚类）的领地，成本高一个量级；
- ④ 阈值与 shingle 的 k 都是超参——没有普适值，工业界按下游效果消融确定。

调试展示：三个真实踩过的坑

以下三个错误都来自本课脚本开发与审查的实测记录（不是虚构的“教学案例”）。

错误 1：每次运行结果都不一样（复现性 bug）

症状：脚本两次运行的 [2] LSH 候选对 数量/成员不同，教程引用的数字对不上；换一台机器、甚至重开一个终端，结果就漂移。

原因：MinHash 里用了 Python 内建 hash() 对 shingle 字符串做基础哈希——hash() 对 str 按进程加盐（PYTHONHASHSEED 随机化），跨进程不可复现。同一篇文档每次运行得到不同签名 → 分桶结果漂移。（本课审查实测抓到的 bug，脚本 minhash_signature 里的注释就是现场记录。）

解法：换成跨进程稳定的哈希，如 `zlib.crc32`（本课脚本用的）或 `hashlib.md5`：

```
python
```

每次进程启动结果都不同

```
hashes = [hash(s) for s in shingle_set]
```

跨进程稳定

```
import zlib
```

```
hashes = [zlib.crc32(s.encode('utf-8')) & 0x7FFFFFFF for s in shingle_set]
```

> △ 这是"教程数字可复现"的底线：任何要写进文档的实验数字，底层的哈希必须稳定。

错误 2：两篇完全不同的短文档被判为"完全重复"

症状：对两篇互不相关的短文档判重——`jaccard(shingles('hi there'), shingles('ok bye'))` 返回 1.0；语料里一批超短文档互相判重，[5] 的去重结果一步删掉了大片文档。

原因：词数 $< k$ 的文档，`range(len(words) - k + 1)` 为空 $\rightarrow$ shingle 集合是空集；而代码约定"两个空集的 Jaccard = 1.0"（避免除零），于是所有短于 k 个词的文档两两之间都是"完全重复"。实测：`shingles('hi there')` 与 `shingles('ok bye')` 都是 `set()`，Jaccard 恰为 1.0。

解法：在 shingling 之前先过滤超短文档（工业界 `words_num_filter` 的 `min_num` 就是干这个的），或对空集合返回哨兵值而非 1.0：

```
python
```

```
if len(words) < k:
```

```
    return set() # 集合为空
```

调用方：`len(shingle_set) == 0`

的文档直接跳过去重（视为"不可比较"）

错误 3：候选对爆炸，LSH 退化成暴力法

症状：[2] LSH 候选对从 3 对涨到几千对， $n=50$ 的玩具语料跑出 1225 对（ $= C(50,2)$ 全部成对），运行时间随 n^2 增长——LSH 的加速优势消失。

原因：`bands  $\times$  rows` 与签名维度不一致。若签名 64 维但配了 `bands=20`, `rows=4` ($=80 > 64$)，越界的 band 切片切出空 `tuple()`——所有文档在这个 band 上"完全相同"，于是全部两两成为候选（实测复现：1225/1225）。Python 切片不抛 `IndexError`，所以这个错误静默发生。

解法：加一行断言，让配置错误尽早炸出来：

```
python
```

```
assert num_hashes == bands * rows, \
```

```
f"签名维度 {num_hashes}  $\neq$  bands $\times$ rows={bands}*{rows}, LSH 将退化为暴力比较"
```

性能数据（实测参考）

方法	复杂度	10 亿文档耗时	精度
暴力两两比较	$O(N^2)$	~10 ¹⁸ 秒（不可行）	100%
MinHash + LSH	$O(N)$	~3.3M CPU 小时（FineWeb）	~99%
精确去重（hash）	$O(N)$	~1 小时	100%（仅精确）

> 数据来源：FineWeb 论文 + 本课开发机实测

常见陷阱

陷阱 1：shingle 大小选择不当

症状：去重效果不好，或误报太多

原因：shingle 太小（误报多）或太大（漏召回）

解法：文本用 3-5 gram，代码用 token 级别

陷阱 2：band/row 参数选择不当

症状：召回率低，或误报率高

原因：b/r 比例不合适

解法：根据目标阈值反推 b 和 r

陷阱 3：哈希函数数量不够

症状：签名估计不准

原因：k 太小，方差大

解法：k=128-256 是常见配置

最佳实践

FineWeb 的去重配置

参数	值	说明
shingle	5-gram	文本级别
k	128	签名维度
b	14	band 数量
r	8	每 band 行数
阈值	0.7	Jaccard 阈值

学完本章你能...

- 手写四阶段去重管线，解释每阶段的数学性质

- 用 $1-(1-J^r)^b$ 设计"召回-误报"预算（给 J 目标反推 b/r）
- 区分字面重复（MinHash）与语义重复（embedding）的处理边界
- 说出 FineWeb 的具体去重配置并解释为什么

概念检验

Q1: $J=0.9$ 、 $r=4$ 、 $b=16$ 时漏掉这对的概率是多少？ $J=0.3$ 呢？

A: $J=0.9$: $1-(1-0.9^4)^{16} \approx 1-(1-0.656)^{16} \approx 1-0.344^{16} \approx 1.0$ （必然命中）。

$J=0.3$: $1-(1-0.0081)^{16} \approx 12.2\%$ ——低相似对偶尔误报，靠第④步验证挡住。

这就是"高相似必召回、低相似低误报"的选择性。

Q2: 为什么用 64 个不同哈希函数而不是一个哈希取 64 个最小值？

A: 需要的是 64 个"独立随机排列"的最小值估计。取同一哈希的 64 个最小值是高度相关的（几乎来自同一排序），不满足 $P[\text{相等}] = J$ 的独立性前提。工程上用 $(a \cdot h + b) \bmod P$ 的仿射族模拟独立排列。

Q3: MinHash 和 SimHash 有什么区别？什么时候用哪个？

A: MinHash 估计 Jaccard 相似度（集合比较），SimHash 估计余弦相似度（向量比较）。

MinHash 适合文本去重（集合视角），SimHash 适合语义相似度（向量视角）。

FineWeb 用 MinHash，因为文本去重是集合比较问题。

动手实践

练习 1: 实现 MinHash 签名函数

任务：实现一个函数，计算集合的 MinHash 签名。

验收标准：

- [] 输入：集合 S，哈希函数数量 k
- [] 输出：签名向量 $[h_1(S), h_2(S), \dots, h_k(S)]$
- [] 使用 $(a \cdot h + b) \bmod P$ 模拟独立哈希函数

步骤提示：

python

```
def minhash_signature(S, k=64):
```

```
    """
```

Steps:

1. 生成 k 个哈希函数的参数 (a, b)
2. 对每个哈希函数，计算 $\min(h(x) \text{ for } x \text{ in } S)$
3. 返回签名向量

```
    """
```

TODO: Implement

pass

练习 2: 实现 LSH 候选对生成

任务：实现一个函数，从签名向量生成候选对。

验收标准：

- [] 输入：所有文档的签名向量，band 数量 b，每 band 行数 r
- [] 输出：候选对列表
- [] 使用分带机制

步骤提示：

python

```
def lsh_candidates(signatures, b=16, r=4):
    """
    Steps:
    1. 将签名向量分成 b 个 band
    2. 对每个 band, 建立哈希桶
    3. 同一桶内的文档是候选对
    4. 返回去重后的候选对列表
    """
```

TODO: Implement

pass

练习 3: 实现 Jaccard 验证函数

任务：实现一个函数，计算两个集合的 Jaccard 相似度。

验收标准：

- [] 输入：集合 A 和 B
- [] 输出：Jaccard 相似度 (0-1)
- [] 正确处理空集合

步骤提示：

```
python
def jaccard_similarity(A, B):
    """
```

Steps:

1. 计算交集 $|A \cap B|$
 2. 计算并集 $|A \cup B|$
 3. 返回 $|A \cap B| / |A \cup B|$
 4. 处理空集合情况
- ```
"""
```

## TODO: Implement

pass

## 课后作业

完成本章后，去 Assignment 13 完成练习：

[Assignment 13](../../assignments/assignment\_13/)

## 下一步

把这 60 行交给工业版：Data-Juicer 的 YAML 管线、200+ 算子与审计体系。

[02 — Data-Juicer 管线](02\_data\_juicer\_pipeline.md)

## 02\_data\_juicer\_pipeline

# 02 — Data-Juicer：YAML 管线、算子全家桶与审计

- > 01 章的 60 行手写真实语料（万亿 token）上要放大四个数量级——那是 Data-Juicer
- > 的领地：200+ 算子（58 过滤 / 95 清洗改写 / 12 去重，含 Ray 分布式变体）、
- > 配置即代码（YAML 可复现可版本化）、逐算子追踪审计。本章给出可照抄的
- > 最小管线 + 与手写版的逐步对照。

## 学习目标

完成本章后，你将能够：

- 配置 Data-Juicer 的 YAML 管线并理解每个算子的作用
- 理解 常见的数据清洗算子（去重、过滤、打分）
- 设计 一条完整的数据清洗管线
- 对照 FineWeb/Gopher 的真实做法
- 识别 算子顺序、阈值选择等常见陷阱

## 前置知识

必须掌握：

- 01 章：MinHash/LSH 四阶段（本章"工业放大"的对象）

## 理论背景

### 问题引入：为什么需要工业工具？

手写去重算法虽然能跑通，但三个根本限制：

1. 算子有限：只实现了去重，没有质量过滤、格式清洗等
2. 扩展性差：单机单线程，无法处理大规模数据
3. 缺乏审计：没有数据追踪和质量报告

Data-Juicer 通过YAML 配置驱动来弥补：

手写: "60 行代码，单一功能"

Data-Juicer: "YAML 配置，200+ 算子，分布式执行"

- > 类比：手写算法像是手工做菜，Data-Juicer 像是用料理机。
- > 料理机功能更多、效率更高，但你需要知道每个按钮的作用。

## 常见算子分类

| 类别 | 算子示例                       | 作用                                                      |
|----|----------------------------|---------------------------------------------------------|
| 去重 | document_line_deduplicator | 跨文档的行级去重（文档级 MinHash 去重用 document_minhash_deduplicator） |
| 过滤 | words_num_filter           | 按词数过滤                                                   |
| 清洗 | remove_header_mapper       | 移除文档开头的 header（LaTeX 语料）                                |



| 类别 | 算子示例                          | 作用                 |
|----|-------------------------------|--------------------|
| 打分 | llm_quality_score_filter      | 用语言模型估计质量分数，过滤低分样本 |
| 选择 | topk_specified_field_selector | 按指定字段排序选取 top-k 样本 |

代码实现

1. 安装与最小管线（CPU 即可）

```
bash
pip install py-data-juicer # 重依赖（Ray/多模态/audio）是可选 extras
```

一个最小 YAML（等价于手写版"过滤 + 去重"）：

```
yaml
dedup_demo.yaml —— 对照手写版：①
过滤（words_num）② 去重（MinHash）

dataset_path: ./tiny_corpus.jsonl # 每行一条 {"text": "..."} (jsonl, 非 JSON 数组)
export_path: ./dedup_output.jsonl

process:
 • words_num_filter: # 过滤：太短的文档（C4/Gopher 启发式的一种）
lang: en
min_num: 20
max_num: 100000
 • document_minhash_deduplicator: # 与手写 60 行同款数学
tokenization: character # 或 space/punctuation
window_size: 5 # 5-gram shingles（FineWeb 同款）
num_permutations: 112 # 签名维度 = bands × rows = 14 × 8（FineWeb 同款）
jaccard_threshold: 0.7 # FineWeb 的等效阈值
num_bands: 14
num_rows_per_band: 8 # ⚠ 参数名是 num_rows_per_band（不是 num_rows）
```

```
bash
dj-process --config dedup_demo.yaml
产物：dedup_output.jsonl + 逐算子的 stats/
追踪报告（每一步删了多少、为什么）
```

2. 手写 ↔ Data-Juicer 逐步对照（本章核心产出）

| 手写（01 章 60 行）   | Data-Juicer                             | 放大点         |
|-----------------|-----------------------------------------|-------------|
| shingles() 正则分词 | 内置多语种分词（Cython/C++ 加速）                  | 万亿 token 吞吐 |
| 64 维签名循环        | C++ minhash + 矢量化，num_permutations: 256 | 精度与吞吐       |

|               |                                                   |                 |
|---------------|---------------------------------------------------|-----------------|
| 手写（01 章 60 行） | Data-Juicer                                       | 放大点             |
| 单机 dict 分桶    | Ray 分布式 LSH（document_minhash_deduplicator 的分布式变体） | 千节点             |
| keep-first 丢弃 | 簇消解策略 + 可选"保留文本最长的"                               | 质量导向            |
| print 日志      | 逐 op 追踪：每个算子前后样本数、被删样本的 HTML 报告                   | 可审计（数据管线必须可审计！） |

- 最值得学的是"配置即代码"哲学：YAML 管线像代码一样 review/版本化/复现——这正是 Data-Juicer 把 Gopher/C4/FineWeb 式清洗规则做成 50+ 配方（data-juicer-hub, 含 RedPajama/BLOOM 复现）的原因。

### 3. 一条"真实感"的完整管线（照抄即用）

```
yaml
process:
 • clean_html_mapper # 去 HTML 壳
 • fix_unicode_mapper # unicode 归一化
 • language_id_score_filter: {lang: en, min_score: 0.8} # 语种过滤（fastText）
 • alphanumeric_filter: {tokenization: word, min_ratio: 0.7} # 符号比异常
 • word_repetition_filter: {rep_len: 10, max_ratio: 0.6} # 行级复读（LLM 吐复读机的饲料）
 • document_minhash_deduplicator: {...} # 全局模糊去重（01 章）
```

对照 FineWeb 的叙述：抽取 → 语种 → 启发式质量（Gopher 规则：文档长度/符号词比/停用词比）→ 全局 MinHash 去重 → 质量分类器。每一类算子都对应上面一个真实条目。

### 4. 追踪审计

Data-Juicer 提供详细的审计报告：

```
bash
查看审计报告
```

```
cat output/audit.json
```

示例输出（示意格式，实际产物以所装 Data-Juicer 版本为准——字段名/文件位置可能不同）

```
{
 "total_samples": 1000000,
 "after_dedup": 950000,
 "after_filter": 800000,
 "after_clean": 780000,
 "quality_score_mean": 0.72,
 "quality_score_std": 0.15
}
```

- > 上面的 JSON 是教学示意：Data-Juicer 的真实审计产物是 stats/ 目录下的逐算子
- > 追踪报告（每个 op 前后的样本数、被删样本明细），字段结构随版本演进。装好环境后
- > 请以自己跑出来的 stats/ 内容为准——"读一遍自己管线删了什么"正是本节的练习。

## 工程实践

### 调试展示：常见错误与修复

#### 错误 1：数据格式不对

症状：

ValueError: Dataset format not supported

原因：数据格式不是 jsonl/csv/parquet

解法：

bash

#### 转换为 jsonl 格式

```
python -c "
import json
with open('data.txt', 'r') as f:
 lines = f.readlines()
with open('data.jsonl', 'w') as f:
 for line in lines:
 f.write(json.dumps({'text': line.strip()}) + '\n')
"
```

#### 错误 2：算子参数错误

症状：

TypeError: \_\_init\_\_() got an unexpected keyword argument 'xxx'

原因：算子参数名不对

解法：

bash

#### 查看算子文档

```
python -m data_juicer.list_ops
```

#### 错误 3：显存不足

症状：

CUDA out of memory

原因：质量评分模型太大

解法：

yaml

#### 换更小的打分模型（或改走 API，不占本地显存）

• llm\_quality\_score\_filter:  
api\_or\_hf\_model: "Qwen/Qwen2.5-0.5B-Instruct" # 而不是 7B 级大模型  
is\_hf\_model: true # true = 本地 Transformers 加载；走 API 则不占本地显存  
min\_score: 0.5

性能数据（量级参考）

| 数据量    | 算子数 | 耗时    | 输出量     |
|--------|-----|-------|---------|
| 10K 条  | 5   | ~1min | ~8K 条   |
| 1M 条   | 10  | ~1h   | ~800K 条 |
| 100M 条 | 15  | ~10h  | ~80M 条  |
| 1B 条   | 20  | ~100h | ~800M 条 |

> 口径说明：上表为 Data-Juicer 官方 benchmark 数字，未经本机复现，仅量级参考——  
> 实际耗时取决于算子组合、机器配置与并行度，量级（线性扩展）才是可信的部分。

常见陷阱

陷阱 1：算子顺序不当

症状：效果不好，或耗时太长

原因：算子顺序影响效果和效率

解法：轻过滤 → 去重 → 重过滤/清洗（与 § 3 管线顺序、Q1 的 FineWeb 口径一致）：  
先用廉价启发式（语种/词数/符号比）砍掉明显垃圾，再做 MinHash 去重，最后才上  
昂贵的重过滤（LLM 质量打分）。纯"先去重"在大语料上代价高：去重本身就是重算子  
（逐文档 shingle/签名/分桶），对未过滤的原始语料全套跑一遍等于给垃圾也建签名；  
而且垃圾重复簇去重后仍会留下"幸存副本"，这些副本照样要各跑一遍重过滤器——  
重复内容浪费算力跑重过滤，正是轻过滤前置要省掉的部分。

陷阱 2：阈值选择不当

症状：过滤太多或太少

原因：阈值太严格（过滤太多）或太宽松（过滤太少）

解法：先用宽松阈值，再逐步收紧

陷阱 3：缺乏审计

症状：不知道数据质量如何

原因：没有开启审计功能

解法：开启审计，查看质量报告

最佳实践

## 管线设计原则

1. 先轻过滤：廉价启发式（语种/词数/符号比）砍掉明显垃圾——便宜，且能整簇去掉垃圾重复
2. 再去重：在缩小后的语料上做 MinHash 去重，签名/分桶的计算量随之变小
3. 后重过滤/清洗：LLM 质量打分、格式归一化等昂贵算子放在最后（FineWeb 口径，同 Q1）
4. 审计贯穿：每步都记录质量指标

## FineWeb 的配置

| 参数   | 值      | 说明             |
|------|--------|----------------|
| 去重阈值 | 0.7    | 5-gram MinHash |
| 语言阈值 | 0.65   | fasttext 语言识别  |
| 质量阈值 | 0.5    | fasttext 质量评分  |
| 最小词数 | 50     | 过滤太短的文档        |
| 最大词数 | 100000 | 过滤太长的文档        |

## 学完本部分你能...

- 用 YAML 搭起"清洗→过滤→去重"的完整管线并解读追踪报告
- 把 01 章手写算法映射到 Data-Juicer 的算子与参数（window\_size/num\_permutations/threshold）
- 说出"配置即代码 + 逐 op 审计"为什么是数据管线的工程底线
- 按 FineWeb/Gopher 的配方思路为自己的语料设计清洗规则

### 概念检验

Q1: 为什么去重要放在质量过滤之后？顺序换一下会怎样？

A: 先去重可省后续计算（同文档只算一次）；但质量过滤可能把"重复簇"删得只剩不同副本，导致本应整体丢弃的低质量重复被保留一份。主流做法：轻过滤 → 去重 → 重过滤/质量打分（FineWeb 的顺序），两个方向都有流派，关键是消融证明。

Q2: num\_permutations 从 64 提到 256，代价和收益各是什么？

A: 签名计算与内存  $\times 4$ ；Jaccard 估计方差更小 → LSH 命中更稳定、阈值附近的行为更平滑。工业界 128-256 是常见档位；再高收益边际递减。

Q3: 如何评估数据清洗的效果？

A: 三种方法：

1. 下游任务效果：用清洗后的数据训练模型，看基准分数
2. 数据质量指标：看质量分数分布、重复率、语言分布等
3. 人工抽样：随机抽 100 条，人工检查质量

### 动手实践

练习 1: 设计一条数据清洗管线

任务：为中文数据设计一条 Data-Juicer 管线。

验收标准：

- [] 包含去重、过滤、清洗算子
- [] 参数合理（参考 FineWeb）
- [] 有审计功能

步骤提示：

yaml

设计思路（顺序 = 轻过滤 → 去重 → 重过滤/清洗，FineWeb 口径）：

1. 语言过滤：language\_id\_score\_filter (zh)
2. 长度过滤：words\_num\_filter
3. 去重：document\_line\_deduplicator（文档级用 document\_minhash\_deduplicator）
4. 质量打分：llm\_quality\_score\_filter
5. 清洗：remove\_header\_mapper

练习 2: 估算管线耗时

任务：估算 100M 条数据的清洗耗时。

验收标准：

- [] 考虑每个算子的复杂度
- [] 考虑并行度
- [] 结果与官方数字接近

步骤提示：

python

```
def estimate_pipeline_time(num_samples, num_ops, parallel=4):
 """
```

Steps:

1. 估算每个算子的耗时
2. 考虑并行度
3. 汇总

```
 """
```

TODO: Implement

pass

## 课后作业

完成本章后，去 Assignment 13 完成练习：

[Assignment 13](../../assignments/assignment\_13/)

下一步

数据准备完、训练完，最后是部署：Part 14 用 vLLM 把模型真正"上线"并量化对比。

[Part 14 vLLM 推理部署](../Part14\_inference\_vllm/tutorial/README.md)